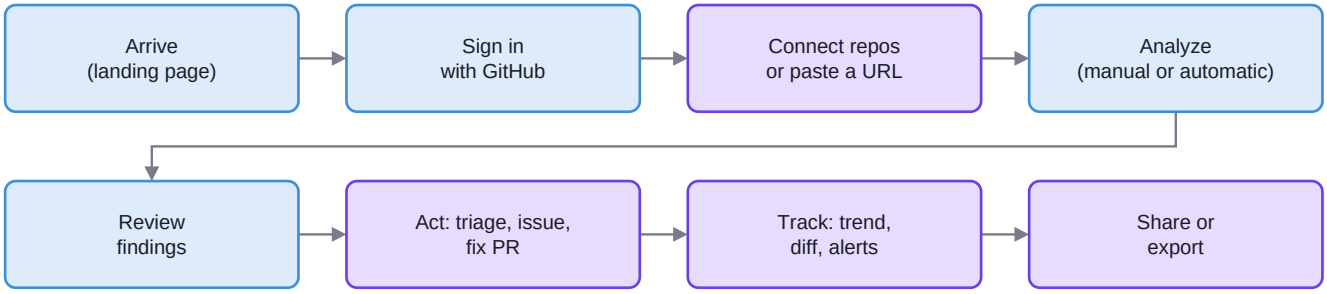
PRISM

Workflows

How people use PRISM and what the system does at each step, from first sign-in to sharing a report. This is the workflow only: no build plan and no implementation detail.

Exists today	New (planned)	Decision point	External service	Warning or failure

The whole journey



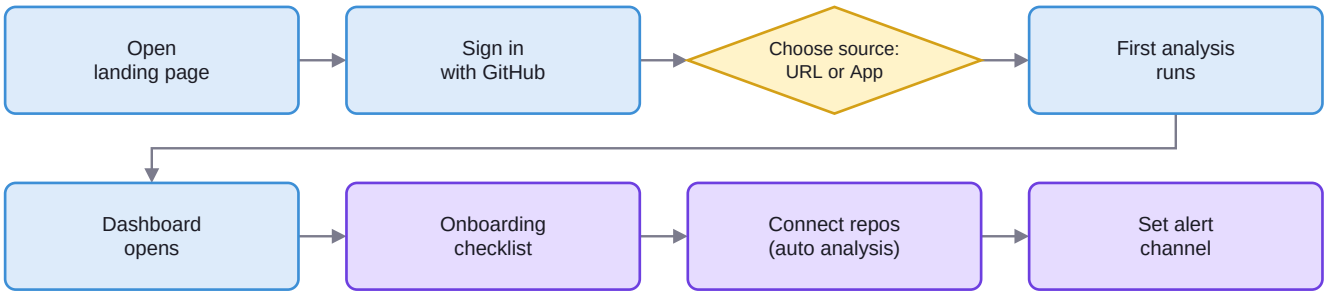
Every workflow below is one part of this journey. Arrive and sign in happen once; connect, analyze, review, act and track repeat for the life of the project. Diagrams show the main path; the branches at each decision point are described in the step table beneath.

The nine workflows

#	Workflow	Who	When	Outcome
W1	First-time setup	New user	First visit	A first analysis, a populated dashboard and a connected repository.
W2	Analyze a repository manually	Any user	Any time	A saved analysis and a dashboard for the repository.
W3	Automatic analysis on push and pull request	Developer (GitHub)	Every push to the default branch and every pull request update	A check and a single comment on the pull request, a saved analysis, and alerts if needed.
W4	Review and act on a finding	Developer	After an analysis, from any issue list	Each finding is fixed, tracked or consciously set aside, and the score reflects it.
W5	History, trend and diff	Developer or lead	To see what changed between two runs	A clear list of new, fixed and unchanged issues and the score movement.
W6	Team lead weekly review	Team lead or maintainer	Weekly	Attention goes to the repositories that got worse; owners are assigned.
W7	Export and share a report	Any signed-in user	When results must leave PRISM	A file or a read-only link, with secrets masked.
W8	Plan limits and billing	User or team owner	When a limit is reached or a plan changes	The right features are on, and nothing is lost when a plan ends.
W9	Failure and recovery	System, then user	When an analysis cannot complete	The user always knows what happened and what to do; transient problems fix themselves.

W1 First-time setup

Who	When	Outcome
New user	First visit	A first analysis, a populated dashboard and a connected repository.



Step by step

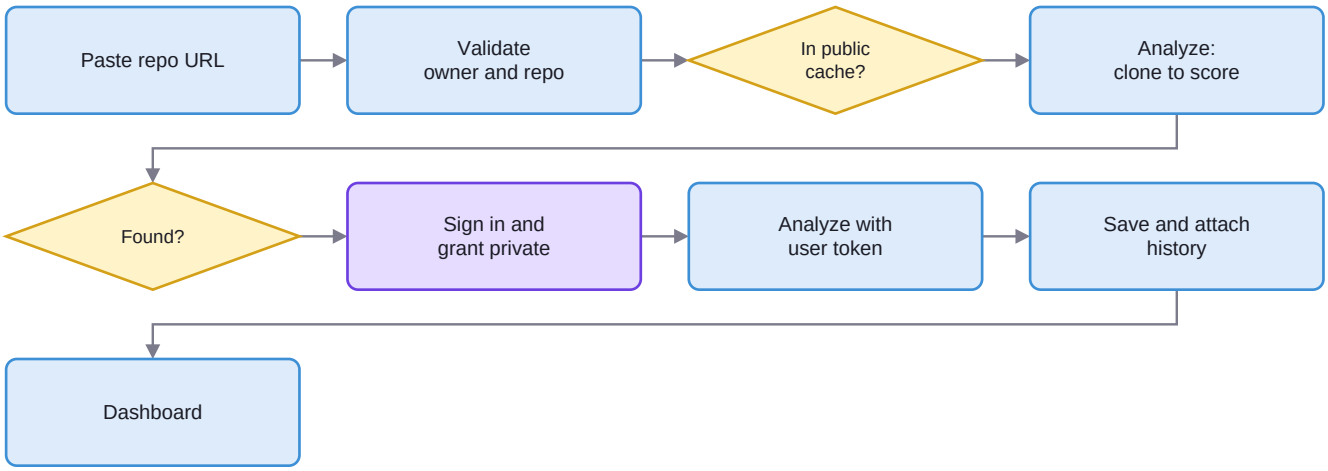
#	Who	What happens
1	User	Opens the landing page and presses Open app.
2	PRISM	Starts GitHub sign-in asking only for the public profile. GitHub's account picker is shown every time.
3	User	Chooses a source: paste a public repository URL, or install the GitHub App to connect repositories for automatic analysis.
4	PRISM	Runs the analysis (clone, parse, secrets, graph, dependencies, quality, score) with a progress screen.
5	PRISM	Opens the dashboard: health score, 3D map, top risks and the AI summary.
6	PRISM	Shows a three-step checklist: connect repositories, triage the first findings, add an alert channel. Each step can be skipped.
7	User	Connects repositories and chooses which ones get automatic analysis.
8	User	Adds a Slack or email channel and sends a test message.

What can go wrong, and what the user sees

- Sign-in cancelled or expired link: back to Setup with a plain message, nothing is stored.
- Repository not found: offer sign-in (private repo) or a spelling check.
- GitHub rate limit: show when it resets; the user can try again then.

W2 Analyze a repository manually

Who	When	Outcome
Any user	Any time	A saved analysis and a dashboard for the repository.



Step by step

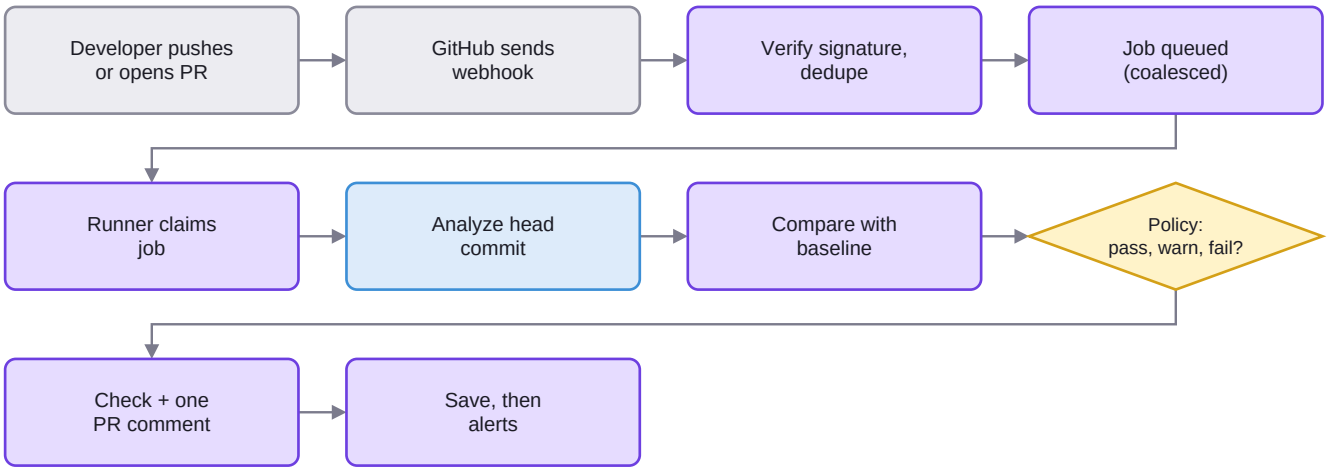
#	Who	What happens
1	User	Pastes owner/repo or a GitHub URL on the New analysis page.
2	PRISM	Checks the name format. A bad name shows a friendly error immediately.
3	PRISM	Looks in the shared public cache (kept for hours). A hit opens the dashboard at once.
4	PRISM	On a miss, downloads the default branch and runs all analysis stages.
5	PRISM	If GitHub says not found, the repo may be private: signed-out users are asked to sign in; signed-in users without the repo scope are asked to allow private access.
6	User	Allows private access (a separate GitHub consent screen). PRISM never stores the token in the browser.
7	PRISM	Analyzes with the user's own token. Private results never enter the shared cache; a result under 30 minutes old is reused.
8	PRISM	Signed-in users get the analysis saved and the last scores attached for the trend.
9	PRISM	Opens the dashboard.

What can go wrong, and what the user sees

- Repo over the size limits: explain the limits and what was skipped.
- No supported source files: say which languages are supported today.
- Dependency check unavailable: analysis continues, with a note.

W3 Automatic analysis on push and pull request

Who	When	Outcome
Developer (GitHub)	Every push to the default branch and every pull request update	A check and a single comment on the pull request, a saved analysis, and alerts if needed.



Step by step

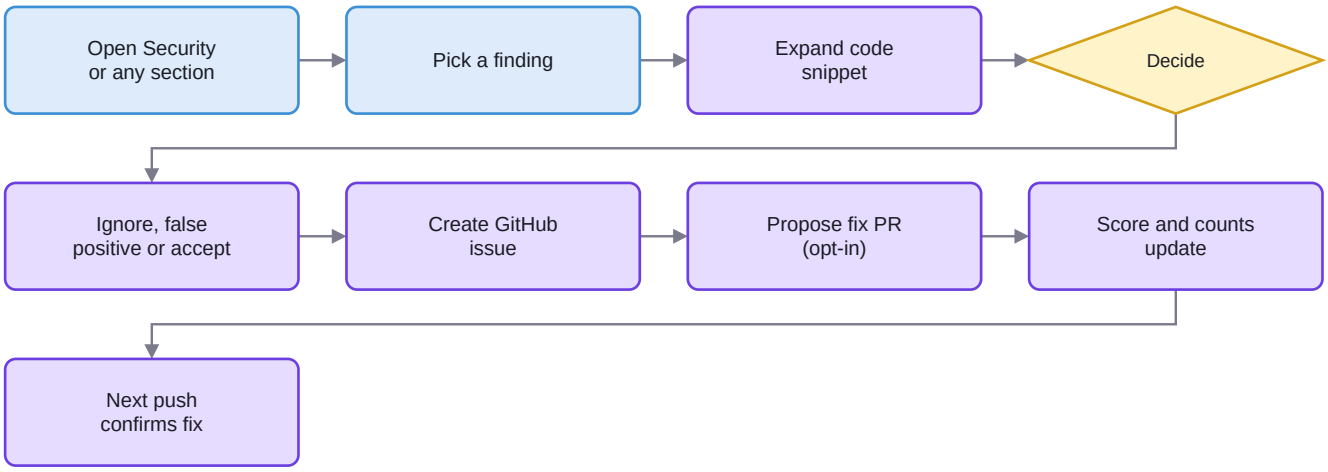
#	Who	What happens
1	Developer	Pushes a commit or updates a pull request.
2	GitHub	Sends a signed webhook to PRISM.
3	PRISM	Verifies the signature, ignores a delivery it has already seen, and answers in under a second.
4	PRISM	Queues a job. Older queued jobs for the same branch are canceled, so ten quick pushes cost one analysis.
5	PRISM	A runner (started every minute) claims the job and marks the PR check as in progress.
6	PRISM	Analyzes the new commit. For a PR it also needs the base commit's analysis; if missing, that is analyzed first.
7	PRISM	Compares the two by issue fingerprint, so only genuinely new problems count. Triaged issues are excluded.
8	PRISM	Applies the repository policy: a new secret or new critical fails; a new high warns; otherwise pass.
9	PRISM	Completes the check with a title such as 'Health 82 to 76 (-6): 2 new critical', adds annotations on the changed lines, and updates one PR comment.
10	PRISM	Saves the analysis, then sends Slack or email alerts if a rule matches, at most one per repository per hour.

What can go wrong, and what the user sees

- Fork pull request with read-only access: neutral check saying the fork cannot be read.
- Baseline missing after two tries: run without it; the comment says so and the check is neutral.
- GitHub or analysis errors: retried after 1, 5 and 15 minutes; then marked failed and shown in History.

W4 Review and act on a finding

Who	When	Outcome
Developer	After an analysis, from any issue list	Each finding is fixed, tracked or consciously set aside, and the score reflects it.



Step by step

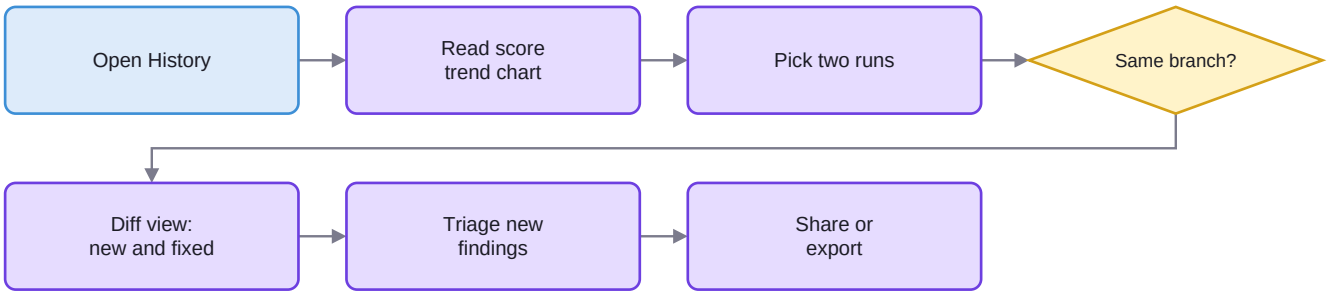
#	Who	What happens
1	User	Opens a section and picks a finding (sorted by risk).
2	PRISM	Expanding it loads the offending lines with context, from the analysed commit. Secret values are always masked.
3	User	Reads the AI explanation and the 'might be a false positive' rating.
4	User	Decides: set it aside (ignore, false positive, accept risk with an expiry), track it (create a GitHub issue), or ask for a fix.
5	PRISM	Set aside: the finding moves to the Triaged tab and the score recalculates immediately. Accepted risk reopens when its expiry passes.
6	PRISM	Track: previews the issue (title, permalink, impact, AI fix) and creates it once; the finding shows 'Tracked: #123'.
7	PRISM	Fix PR (only if the repo allows sending a snippet to the AI): proposes a patch, checks it applies and no longer triggers the rule, then opens a pull request. Never merges.
8	PRISM	On the next analysis, a finding that no longer appears is marked fixed; a linked issue gets a comment, and is closed only if that is switched on.

What can go wrong, and what the user sees

- Signed out: triage and issue buttons are hidden with a hint to sign in.
- Secret findings: no fix PR; the guidance is to rotate the key.
- Triage is per person: it never changes what another user sees.

W5 History, trend and diff

Who	When	Outcome
Developer or lead	To see what changed between two runs	A clear list of new, fixed and unchanged issues and the score movement.



Step by step

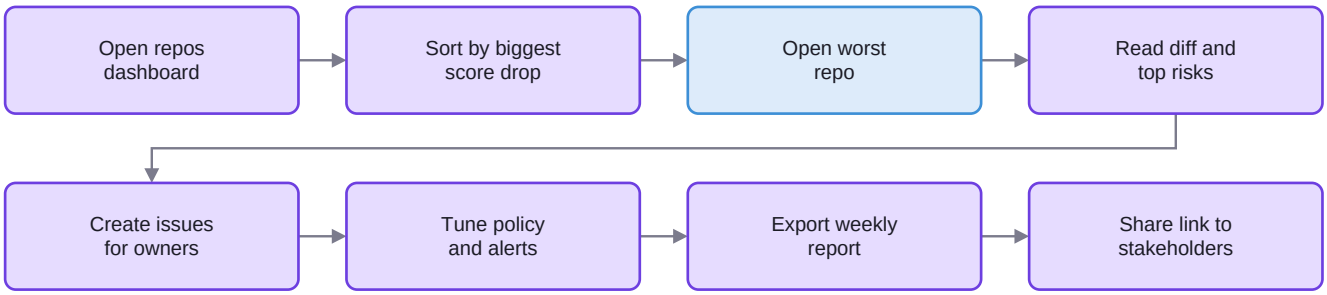
#	Who	What happens
1	User	Opens History and sees a chart of the score over time with a row per run.
2	PRISM	The chart shows the triage-adjusted score by default; a toggle shows the raw score.
3	User	Selects two runs (default: latest against the previous run on the same branch) or presses Compare on a row.
4	PRISM	Checks they are comparable (same branch, scoring version). If not, shows a banner and still compares.
5	PRISM	Shows score change, then New, Fixed, Changed and Unchanged issues and metric changes.
6	User	Triages new findings inline, or opens a finding's snippet.
7	User	Exports the diff or shares the report.

What can go wrong, and what the user sees

- First analysis of a repo: nothing to compare yet.
- Older runs without fingerprints: compared approximately, and labelled so.
- Results capped at 300 issues: the diff says it may be approximate.

W6 Team lead weekly review

Who	When	Outcome
Team lead or maintainer	Weekly	Attention goes to the repositories that got worse; owners are assigned.



Step by step

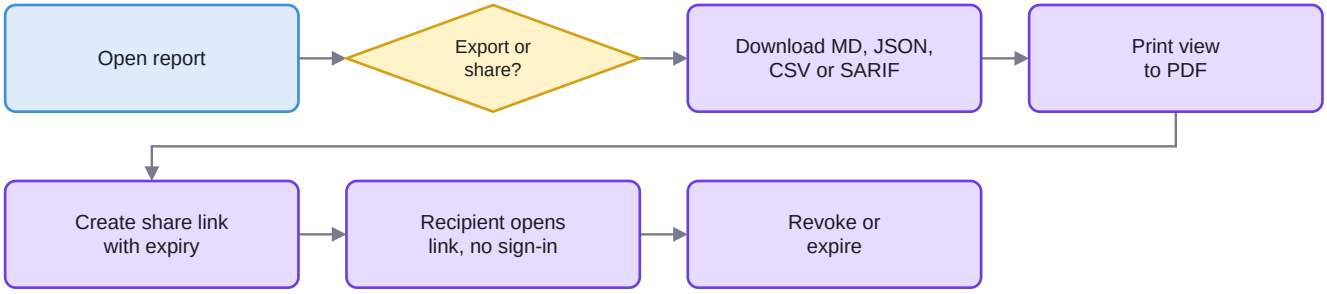
#	Who	What happens
1	User	Opens the repositories dashboard: score, trend, critical count and last-analysed time for every repo.
2	PRISM	Highlights repos below the threshold, not analysed in 30 days, or with a failing run.
3	User	Sorts by biggest score drop this week and opens the worst repository.
4	User	Reads the diff against last week and the top risks and hotspots (complex and frequently changed files).
5	User	Creates GitHub issues for the findings that need an owner.
6	User	Adjusts the repository policy (fail on critical, minimum score, maximum drop) and alert rules, using the preview of how the last ten PRs would have fared.
7	User	Exports a report (print to PDF, Markdown or CSV).
8	User	Creates a share link with an expiry for people who do not use PRISM.

What can go wrong, and what the user sees

- Many repos: the list is paginated and totals are computed in the database.
- Different scoring versions between repos: a banner explains the comparison.

W7 Export and share a report

Who	When	Outcome
Any signed-in user	When results must leave PRISM	A file or a read-only link, with secrets masked.



Step by step

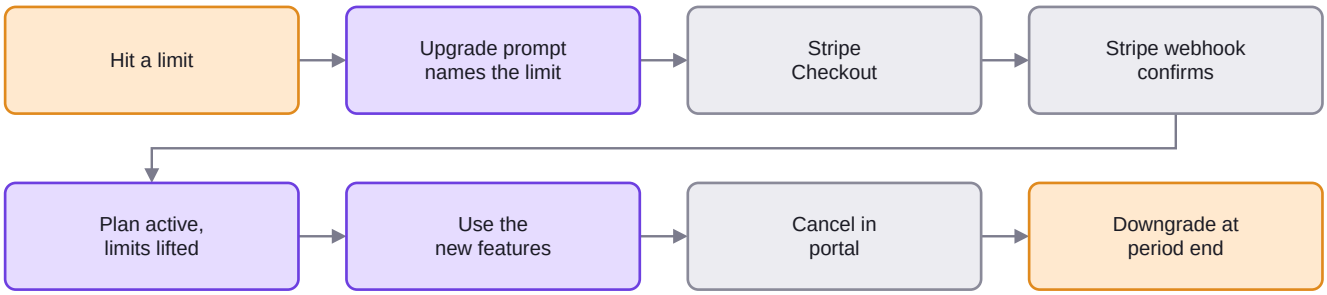
#	Who	What happens
1	User	Opens a repository report and presses Export or Share.
2	User	Export: picks a format. SARIF can also be sent to the GitHub Security tab.
3	PRISM	Builds the file from the saved analysis. Secret values are masked in every format; triaged issues keep their state.
4	User	Print view: opens a print-ready report and saves it as PDF from the browser.
5	User	Share: chooses 24 hours, 7 days or 30 days. For private repos a warning appears and file names are hidden unless ticked.
6	Recipient	Opens the link without signing in and sees a read-only snapshot of that single analysis.
7	User	Revokes the link from the list, or lets it expire. It stops working within about a minute.

What can go wrong, and what the user sees

- Large SARIF (over 10 MB or 25,000 results): trimmed to the highest risks with a note.
- Free plan: share links limited or unavailable, with an upgrade prompt.

W8 Plan limits and billing

Who	When	Outcome
User or team owner	When a limit is reached or a plan changes	The right features are on, and nothing is lost when a plan ends.



Step by step

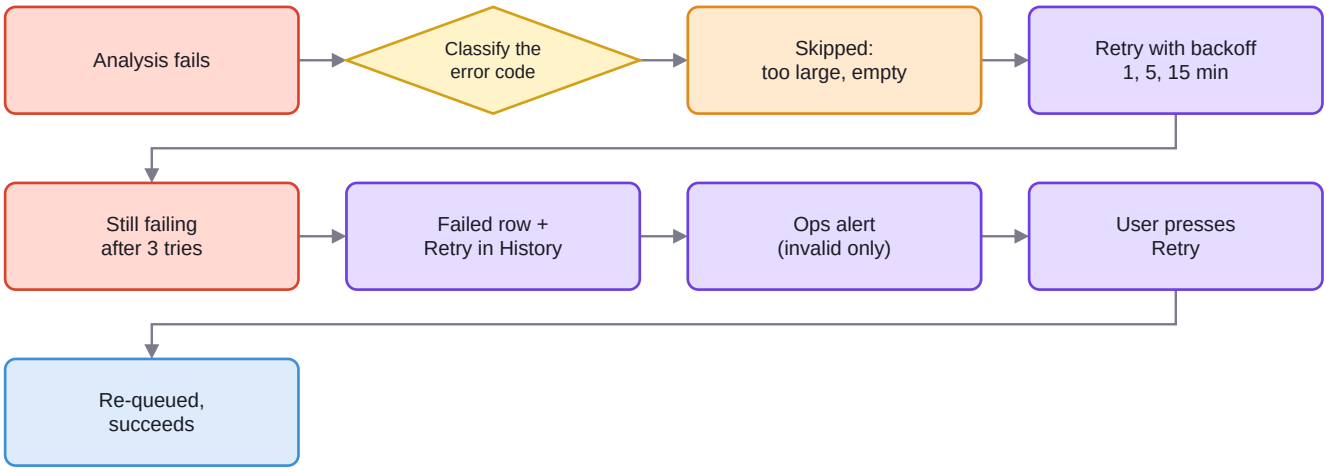
#	Who	What happens
1	User	Reaches a limit (for example a fourth private repository).
2	PRISM	Shows which limit was reached, the current usage and what each plan includes.
3	User	Chooses a plan and pays on Stripe's hosted page.
4	Stripe	Sends a signed event; PRISM verifies it and records the subscription once (replays change nothing).
5	PRISM	Applies the plan immediately; the user returns to the page they started from.
6	User	Uses the features: automatic analysis, PR checks, alerts, more AI messages.
7	User	Cancels or changes the card in Stripe's customer portal.
8	PRISM	At period end returns to Free: automatic analysis pauses and extra private repos become read-only. Nothing is deleted.

What can go wrong, and what the user sees

- Payment fails: the plan stays for 7 days with a banner, then ends.
- Webhook arrives before the user returns: both paths use the same record.

W9 Failure and recovery

Who	When	Outcome
System, then user	When an analysis cannot complete	The user always knows what happened and what to do; transient problems fix themselves.



Step by step

#	Who	What happens
1	PRISM	An analysis fails: GitHub unreachable, rate limited, repo too large or empty, or an unexpected error.
2	PRISM	Classifies it. Too large or empty is final: the run is marked skipped with the reason.
3	PRISM	Network and rate-limit errors are retried automatically after 1, 5 and 15 minutes. A run stuck for over 10 minutes is requeued.
4	PRISM	After the third failure the run is marked failed and appears in History as a red row with the error text.
5	PRISM	Unexpected errors also raise an alert for the operator; the others do not.
6	User	Presses Retry on the failed row once the cause is fixed (for example the rate limit has reset).
7	PRISM	Re-queues it at normal priority; on success the row turns green and the PR check, if any, is updated.

What can go wrong, and what the user sees

- Interactive page views show the same message immediately instead of waiting for retries.
- The shared cache keeps failures for only a few minutes so a fixed problem is not hidden.